

ORDER BY in Window Functions - Complete Notes

Important Mental Model

ORDER BY inside a window function **does logically sort each partition internally** for the purpose of the window function's calculations. However, **it does not sort the rows returned by the query**. The final output order is determined **only by the outer ORDER BY** (or is undefined if there isn't one).

1. Two Types of ORDER BY

There are two completely different uses of ORDER BY in SQL.

A. Normal ORDER BY

```
SELECT *
FROM products
ORDER BY price;
```

Purpose:

- Sorts the **final output**.
- Does **not** affect calculations.

B. Window ORDER BY

```
SELECT title, price,
ROW_NUMBER() OVER(ORDER BY price)
FROM products;
```

Purpose:

1. Defines the logical order of rows inside each partition.
2. Changes the default window frame.

It does not sort the final output.

2. Window ORDER BY does NOT sort the output

```
SELECT prod_id, price,
ROW_NUMBER() OVER(ORDER BY price) AS rn
FROM products
ORDER BY prod_id;
```

Step 1: Window function logically orders rows by price and assigns row numbers.

Step 2: The outer ORDER BY sorts the final displayed result.

Window ORDER BY controls calculations.

Outer ORDER BY controls presentation.

3. ORDER BY defines the logical order

Example partition prices: 23.99, 9.99, 18.99.

ORDER BY price internally treats them as:

9.99 → 18.99 → 23.99.

Functions using this order include FIRST_VALUE, LAST_VALUE, LAG, LEAD, ROW_NUMBER, RANK, DENSE_RANK and NTILE.

4. ORDER BY changes the default frame

Without ORDER BY: Entire partition (equivalent to ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING).

With ORDER BY: Default frame becomes RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW.

5. Effect on Aggregate Functions

```
SUM(price) OVER(PARTITION BY category)
```

Returns the total sum for the entire partition.

```
SUM(price) OVER(PARTITION BY category ORDER BY price)
```

Returns a running total.

6. FIRST_VALUE()

```
FIRST_VALUE(price) OVER(PARTITION BY category ORDER BY price)
```

Sorts each category by price and returns the first price. This is equivalent to MIN(price)

OVER(PARTITION BY category) **only because** the ordering column and value column are both price.

7. LAST_VALUE()

```
LAST_VALUE(price) OVER(PARTITION BY category ORDER BY price)
```

Without an explicit frame, LAST_VALUE returns the current row's value because the default frame ends at CURRENT ROW.

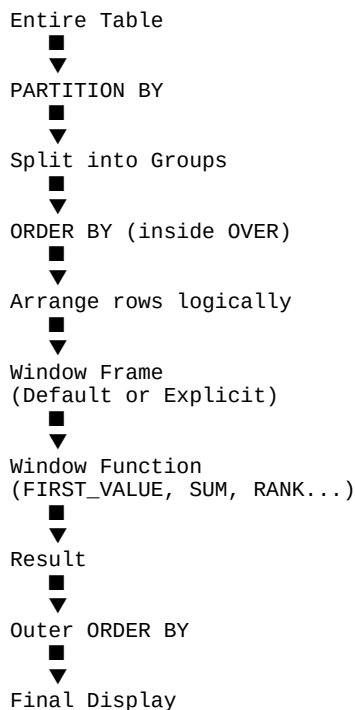
```
LAST_VALUE(price) OVER(PARTITION BY category ORDER BY price RANGE BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING)
```

With UNBOUNDED FOLLOWING, LAST_VALUE sees the entire partition and returns the true last value.

8. FIRST_VALUE vs MIN

FIRST_VALUE(price ORDER BY price) = MIN(price) only in this specific case. If you ORDER BY release_date, FIRST_VALUE(price) returns the price of the earliest released product, not the minimum price.

9. Mental Model



Key Takeaways

- Outer ORDER BY sorts the final output.
- Window ORDER BY does not sort the final output.

- It logically orders rows within each partition.
- It changes the default frame to UNBOUNDED PRECEDING ... CURRENT ROW.
- FIRST_VALUE(price ORDER BY price) equals MIN(price) only in that specific scenario.
- LAST_VALUE usually needs UNBOUNDED FOLLOWING.

Golden Rule:

ORDER BY inside OVER() controls the calculation.

ORDER BY outside OVER() controls the presentation.